Relatório técnico: FakenewsBR v6 + BERTimbau

Detecção de desinformação em português (pt-BR e pt-PT) por classificação binária de texto: `fake` (1) × `true` (0).

Modelo final: BERTimbau base (`neuralmind/bert-base-portuguese-cased`) ajustado (fine-tuning) sobre o pool v6 deduplicado, com pesos DFR por célula grupo×rótulo e calibração Platt. Nome do run: `D1_dedup_s42`.

Resultado no teste (n = 13.661, limiar 0,5):

acurácia	macro-F1	F1 fake	F1 true	acurácia da classe fake	acurácia da classe true	pior grupo (macro-F1)	ECE
0,8251	**0,7850**	0,8779	0,6920	0,8575	0,7361	**0,6173**	0,0236

Supera o baseline TF-IDF + regressão logística (0,8032 / 0,7404 / pior grupo 0,5070) e o modelo v1 anterior, mesmo no mesmo teste. A classe **true** é o ponto fraco: o modelo acerta 74% das notícias verdadeiras, contra 86% das falsas.

Sumário

1. Preparação dos dados
2. Treinamento do modelo
3. Critérios de seleção da abordagem
4. Métricas e análise dos resultados
5. Principais desafios
6. Aprendizados e lições
7. Limitações e próximos passos
8. Reprodução

---

1. Preparação dos dados

1.1 Origem e evolução do dataset

O dataset foi construído em versões sucessivas. A v1 está preservada byte a byte em todas as versões seguintes.

versão	linhas	o que entrou
v1	39.466	base histórica (framework AKCIT-FN)
v2	215.640	agências de checagem, portais, G1, feed ClaimReview
v3	242.913	LIAR-BR e AVERITEC-BR traduzidos

versão	linhas	o que entrou
v4	291.521	Polígrafo (pt-PT, cerca de 12,1 mil artigos) e camadas de verificação
v5	296.377	Google Fact Check API por publisher (Aos Fatos, Estadão, Observador, Comprova, UOL)
v6	**297.672**	AFP Checamos, Pública, Tatu, Aletheia; rótulo `unknown` explícito para o que não foi validado

1.2 Rótulos em camadas: o que pode entrar no treino

Cada linha recebe uma camada de rótulo (`label_tier`) que indica **quão confiável** é o rótulo. Só as camadas verificadas entram no treino supervisionado.

camada	linhas	significado	entra no treino?
`checker`	44.347	veredito de agência de checagem ou corpus externo checado	sim
`v1`	39.466	rótulo da base histórica	sim
`checker_match`	4.166	correspondência ≥ 0,9 com uma checagem ClaimReview	sim
`llm_local`	3.092	verificação automática com evidência e confiança ≥ 0,8	sim
`corroborated`	9	≥ 3 portais independentes	sim
`provenance`	206.592	manchete considerada `true` só pela origem, **sem verificação**	**não**

Resultado: um **pool de treino de 91.080 linhas** (66.772 fake / 24.308 true, ou seja, 73,31% fake). As 206.592 linhas sem verificação nunca entram no treino.

1.3 O problema central: atalho de procedência

O pool reúne 36 grupos de origem. **11 deles (39.373 linhas, 43% do pool) têm rótulo constante**: por exemplo, `fakes` (20.347 linhas, 100% fake), `FC_BOATOS` (10.772, 100% fake) e `true` (2.710, 100% true). Um modelo treinado ingenuamente aprende a reconhecer **a origem** do texto (estilo de agência de checagem, formato de manchete), e não a veracidade.

Um diagnóstico deixou isso concreto: prever o grupo de origem a partir do texto e responder com a proporção de fake daquele grupo, sem olhar o conteúdo, já dá **acurácia 0,8055 e macro-F1 0,7474**. Mas o **pior grupo cai para 0,2717**. Esse é o teto de quem trapaceia, e a acurácia média sozinha não detecta a trapaça.

Só 8 grupos são **informativos**, isto é, têm as duas classes em proporção razoável (n ≥ 200 e minoria ≥ 15%): Polígrafo, Fake.br, LIAR-BR, FakeWhatsApp.BR, AVERITEC-BR,

COVID19.BR, COVID19.BR_raw e LLM4BR_300. Juntos somam 36.896 linhas (51,98% fake). É neles que a pergunta "o texto é verdadeiro?" está bem posta. Por isso viraram a base da métrica de pior grupo e da calibração.

1.4 Limpeza e higiene

etapa	o que foi feito	números
Texto de entrada	coluna `text_no_url` (texto sem URLs)	—
Caracteres corrompidos	linhas com `U+FFFD` marcadas e removidas do treino e da avaliação	4 linhas (3 nos splits usados)
Mojibake	marcadas para auditoria, mantidas	34 linhas
Duplicatas exatas (removidas)	texto normalizado (sem acento, só `[a-z0-9 ]`, espaços colapsados); fica **uma linha por texto**: a do rótulo majoritário e, entre elas, a de menor `rid` (`dedup_exatas.py`)	427 textos repetidos (900 linhas) → **473 cópias removidas**, todas do treino (−315 fake, −158 true); pool 91.080 → **90.607**, com **0 duplicatas exatas** restantes. 3 textos tinham rótulo conflitante: 1 resolvido pela maioria (3 fake × 1 true → fake) e 2 empates (1 × 1) decididos pelo menor `rid`
Quase-duplicatas (agrupadas, não removidas)	MinHash (k = 64, LSH 16×4) sobre 5-gramas de palavras, verificação exata de Jaccard ≥ 0,8, union-find	85.592 clusters; 4.454 com mais de uma linha (até 22 linhas); cada cluster fica inteiro de um lado do split (seção 1.5)

1.5 Divisão treino / validação / teste

A divisão é feita **por cluster de quase-duplicatas**: todas as cópias e variantes próximas de um texto caem do mesmo lado. A atribuição é estratificada por rótulo e por grupo (desvio de cerca de 1 p.p.). Depois da divisão, uma varredura exata confirma **0 pares com Jaccard ≥ 0,8** entre treino e teste e entre treino e validação (máximo observado: 0,798).

Split usado: `full_iid` (pool completo, 70 / 10 / 5 / 15):

parte	papel	n (após dedup e U+FFFD)	% fake
`train`	ajuste dos pesos	63.281	73,36
`val_sel`	escolha da época e do run (early stopping)	9.108	73,31
`val_calib`	calibração Platt e limiar alternativo	4.554	73,30
`test`	**só para o relatório final**	13.661	73,31

A deduplicação removeu linhas **só do treino**: o teste é idêntico ao dos runs anteriores, o que mantém a comparação justa.

Há também dois splits auxiliares preparados: `ood_wa` (teste = todo o canal WhatsApp, para generalização entre canais) e `bal_iid` (só os grupos informativos).

---

2. Treinamento do modelo

2.1 Arquitetura e hiperparâmetros

item	valor
Modelo base	`neuralmind/bert-base-portuguese-cased`, 12 camadas, 108,9M parâmetros
Cabeça	`BertForSequenceClassification`, 2 classes
Congelamento	embeddings + 6 primeiras camadas (43,1M parâmetros treináveis)
Comprimento máximo	192 tokens (média no treino: 48,7; cerca de 9% dos textos são truncados)
Otimizador	AdamW, lr 2e-5, weight decay 0,01, gradient clipping 1,0
Agenda	warmup linear de 400 passos, batch 32 (1.978 passos por época)
Épocas	teto de 15, com **early stopping** (paciência 1, ganho mínimo 0,005 de macro-F1 em `val_sel`)
Batches	agrupados por comprimento, para pagar menos padding
Precisão	AMP bf16
Reprodutibilidade	seed 42, `cudnn.deterministic=True`, SHA-256 dos dados e do script registrados no run

2.2 Pesos DFR (anti-atalho)

Cada exemplo recebe peso **1 / tamanho da sua célula grupo×rótulo**, normalizado para média 1. Assim, uma célula gigante como `fakes|fake` (cerca de 14 mil linhas) não domina a perda. Células minúsculas (há células com 1 exemplo) teriam peso de até cerca de 1.037, por isso o peso é **cortado em 25** e renormalizado (peso final máximo: 34,1).

Efeito: a proporção de fake que a função de perda efetivamente vê cai de 73% para **0,60**. O modelo deixa de ganhar "de graça" ao prever a classe majoritária e é empurrado para os grupos informativos.

2.3 Calibração e limiar

O score bruto do modelo não é uma probabilidade confiável. A calibração **Platt** ajusta `p = sigmoid(a·z + b)`, em que `z = logit_fake − logit_true`. O ajuste é feito em `val_calib` **restrito aos grupos informativos** (n = 1.844, prior de 52% fake): calibrar na validação inteira (73% fake) faria o intercepto reproduzir esse prior artificial e desfaria o DFR.

	valor
a, b	0,4861 / 0,0198
ECE em `val_calib`	0,0716 → **0,0197**
limiar primário	0,5
limiar ótimo na validação	0,46 (macro-F1 0,7494 em `val_calib`)

2.4 Execução do run final

item	valor
Hardware	NVIDIA RTX 3060 Laptop (CUDA 13.0, torch 2.13, transformers 5.16)
Tempo total	645,7 s (cerca de 11 min); treino 563,5 s
Vazão	cerca de 23 mil tokens/s; pico de VRAM de 2,2 GB
Épocas	4 rodadas, **melhor = 3** (early stopping na 4)

Curva de treino:

época	perda treino	macro-F1 treino	perda val	macro-F1 val	pior grupo val	ECE val
1	0,511	0,747	0,397	0,733	0,499	0,034
2	0,381	0,786	0,498	0,755	0,543	0,100
3	0,280	0,853	0,407	**0,783**	**0,600**	0,071
4	0,191	0,886	0,483	0,778	0,581	0,095

Na época 4 a perda de treino continua caindo, mas a validação piora: é o início do overfitting, e o early stopping parou no ponto certo.

O run foi **refeito do zero** com a mesma seed (como `D1_dedup_s42_perclass`) e reproduziu exatamente as mesmas métricas, o que confirma o determinismo.

---

3. Critérios de seleção da abordagem

3.1 Princípios que guiaram as escolhas

1. **Robustez acima de média.** A métrica-manchete é o **macro-F1 do pior grupo confiável**, não a acurácia. Um modelo que acerta 82% na média, mas zera um grupo, está explorando atalhos.
2. **Escolher na validação, nunca no teste.** Época e configuração são escolhidas pelo macro-F1 em `val_sel`. O teste só é aberto para o relatório.
3. **Melhoria só conta acima do ruído.** A variação natural entre seeds da configuração BASE (amplitude do macro-F1 de validação em 3 seeds) foi **0,0038**. Uma variante só é considerada melhor se superar isso e confirmar em 3 seeds.
4. **A régua honesta é um baseline forte.** O BERT só se justifica se superar o TF-IDF + regressão logística, principalmente no pior grupo.

3.2 Por que BERTimbau ajustado

alternativa	resultado no teste	decisão
Sempre prever "fake"	acc 0,7331, macro-F1 0,4230	piso trivial
Prior da origem (trapaceiro)[1]	acc 0,8055, macro-F1 0,7474, pior grupo **0,2717**	mostra que acurácia alta não prova nada
TF-IDF + regressão logística + Platt	acc 0,8032, macro-F1 0,7404, pior grupo 0,5070	baseline linear forte
Encoder congelado + cabeça linear (pipeline anterior)	ajudou a medir o atalho, mas o ajuste fino rendeu mais fora de domínio	substituído
BERTimbau ajustado + DFR + Platt	acc 0,8251, macro-F1 0,7850, pior grupo **0,6173**	**adotado**

[1] Medido por `models/v6/diagnose_source_leak.py` no teste v6 da divisão anterior (antes do split por cluster). As demais linhas usam exatamente o mesmo teste de 13.661 linhas.

O BERTimbau é pré-treinado em português brasileiro e tem custo viável em uma GPU de notebook (cerca de 11 min por run). O XLM-RoBERTa foi considerado por causa da queda em pt-PT (seção 4.6), mas não foi testado nesta rodada.

3.3 Experimentos de hiperparâmetros

Primeiro, uma busca rápida exploratória ("autoresearch": 23 runs curtos e 4 controles) indicou as direções. Uma revisão crítica dela encontrou dois erros de desenho (orçamento medido em passos, e não em tokens, e épocas diferentes entre os eixos comparados), e por isso **ela só orientou, não decidiu**.

Depois vieram os runs completos, cada variação contra a BASE (ordenados pelo macro-F1 de validação, média das seeds):

teste	o que muda	seeds	macro-F1 val	Δ vs BASE	pior grupo val	min/run	macro-F1 teste	pior grupo teste
T4_freeze3	congela 3 camadas (não 6)	3	0,7842	+0,0019	0,576	14,7	0,7870	0,5972
T3_patience2	paciência 2	1	0,7839	+0,0017	0,557	13,8	0,7839	0,6100
T2_lr5e-5	lr 5e-5	3	0,7830	+0,0008	0,548	11,8	0,7837	0,6014
D1_dedup	**sem as 473 cópias exatas**	1	0,7828	+0,0005	**0,600**	10,8	**0,7850**	**0,6173**
BASE	config. de referência	3	0,7823	—	0,550	10,2	0,7850	0,6039
T7_clip10	corte dos pesos DFR em 10	1	0,7814	−0,0008	0,550	10,6	0,7849	0,6076
T1_lr3e-5	lr 3e-5	1	0,7810	−0,0013	0,589	10,6	0,7826	0,6156
T6_maxlen256	256 tokens	3	0,7791	−0,0032	0,525	15,2	0,7844	0,6019
T9_freeze0_lr1e-5	encoder inteiro, lr 1e-5	1	0,7751	−0,0072	0,571	17,8	0,7796	0,5809
T5_freeze0	encoder inteiro	1	0,7735	−0,0087	0,547	11,0	0,7704	0,5544
T8_droptiers	sem `llm_local` e `corroborated`	1	0,7553	**−0,0270**	0,550	5,5	0,7509	0,5729

Leitura:

- **Nenhuma variante supera a BASE acima do ruído** (0,0038) na validação. A configuração BASE já está num platô.
- **Pioram de forma clara:** descongelar o encoder inteiro (T5, T9) e remover as camadas de rótulo `llm_local` e `corroborated` (T8, −0,027). Essas camadas trazem quase todos os exemplos `true` de portais, e sem eles o modelo perde a classe minoritária.
- **Empatam, mas custam mais:** congelar menos camadas (T4, +45% de tempo) e 256 tokens (T6, +50% de tempo). Por isso foram mantidos `freeze_layers=6` e

`max_length=192`.

- **D1_dedup foi adotado como modelo final.** Empata com a BASE na validação, tem o melhor pior grupo de validação da tabela (0,600) e é **metodologicamente mais limpo**: sem cópias exatas, nenhum texto ganha peso dobrado no treino. A escolha se apoia na higiene dos dados, e não no número do teste. Ele tem só uma seed, e a confirmação em 3 seeds está nos próximos passos.

---

4. Métricas e análise dos resultados

4.1 Métricas consideradas

métrica	o que mede	por que importa aqui
Acurácia	fração de acertos	intuitiva, mas inflada pela classe majoritária (73% fake)
Macro-F1	média do F1 das duas classes	dá o mesmo peso a fake e true
F1, precisão e **acurácia por classe**	desempenho separado de fake e de true	a "acurácia da classe" é a fração dos exemplos daquela classe que o modelo acertou (= recall)
PR-AUC / ROC-AUC	qualidade do ranking, independente de limiar	compara modelos sem escolher corte
Brier / **ECE**	calibração (a probabilidade de 0,8 acerta 80% das vezes?)	o score será lido como probabilidade
Pior grupo (macro-F1)	o grupo confiável em que o modelo vai pior	detecta atalho de procedência; só conta grupos com n ≥ 100 e minoria ≥ 20
Recortes	canal, era, dialeto, camada de rótulo, casos limítrofes	mostra onde o modelo falha

4.2 Resultado global e por classe

Teste `full_iid`, n = 13.661 (10.015 fake / 3.646 true):

limiar	acc	macro-F1	PR-AUC	ROC-AUC	Brier	ECE	pior grupo
0,5	**0,8251**	**0,7850**	0,9586	0,8956	0,1153	0,0236	**0,6173**
0,46 (ótimo na validação)	0,8277	0,7802	0,9586	0,8956	0,1153	0,0236	0,5891

limiar	classe	n	acurácia da classe	precisão	F1
0,5	fake	10.015	0,8575	0,8993	0,8779

limiar	classe	n	acurácia da classe	precisão	F1
0,5	**true**	3.646	**0,7361**	0,6529	**0,6920**
0,46	fake	10.015	0,8814	0,8833	0,8823
0,46	true	3.646	0,6802	0,6761	0,6782

Análise:

- **A classe true é a mais difícil.** O F1 de true (0,69) fica 19 pontos abaixo do de fake (0,88). Além do desequilíbrio (27% true), a precisão de 0,65 indica que, quando o modelo diz "verdadeiro", erra em 35% dos casos.
- **O limiar é uma troca entre as classes.** Baixar o corte para 0,46 aumenta o acerto de fake (+2,4 p.p.) às custas de true (−5,6 p.p.), e piora o macro-F1 e o pior grupo. Por isso **0,5 foi mantido como limiar primário**.
- **O ranking é bom** (PR-AUC 0,96, ROC-AUC 0,90) **e a calibração também** (ECE 0,024; antes do Platt era 0,068).

4.3 Comparação com os modelos anteriores (mesmo teste)

modelo	acc	macro-F1	F1 true	acurácia true	pior grupo	ECE
Maioria (tudo fake)	0,7331	0,4230	0,0000	0,0000	0,2717	0,2669
TF-IDF + regressão logística + Platt	0,8032	0,7404	0,6126	0,5828	0,5070	0,0468
v1 antigo (30,4% do teste estava no treino dele)	0,8051	0,6943	0,5103	0,3804	0,3008	0,0625
v1 antigo, só a parte limpa (n = 9.514)	0,7650	0,5974	0,3377	0,2320	0,3008	0,0959
BERTimbau v6 R0 (antes do dedup)	0,8061	0,7675	0,6728	**0,7468**	0,5410	0,0315
BERTimbau v6 D1_dedup (final)	**0,8251**	**0,7850**	**0,6920**	0,7361	**0,6173**	**0,0236**

O modelo final é o melhor em quase tudo. Em relação ao baseline linear: **+0,045 de macro-F1, +0,11 no pior grupo, +15 p.p. de acurácia em true e metade do ECE**. A única métrica em que o R0 anterior é melhor é a acurácia de true (−1,1 p.p. no final), porque o modelo final passou a acertar mais fakes.

O v1 antigo mostra como a contaminação engana: no teste contaminado ele parece competitivo (acc 0,805), mas na parte limpa acerta só 23% das notícias verdadeiras.

4.4 Desempenho por grupo de origem

Grupos confiáveis do teste (os que contam para o pior grupo), do pior para o melhor:

grupo	n	% fake	acc	acc fake	acc true	macro-F1	F1 true
EXT_LIARBR	1.049	40,3	0,6215	0,6407	0,6086	**0,6173**	0,6575
FC_POLIGRAFO (pt-PT)	1.625	61,7	0,6338	**0,5055**	0,8408	0,6338	0,6374
FC_EFARSAS	481	87,1	0,8524	0,9189	**0,4032**	0,6644	0,4132
EXT_AVERITECBR	439	72,9	0,7175	0,7469	0,6387	0,6724	0,5507
FakeWhatsApp.BR_2018	957	37,3	0,7116	0,8431	0,6333	0,7096	0,7336
COVID19.BR	290	43,4	0,7931	0,8810	0,7256	0,7929	0,7987
Fake.br	1.075	56,6	0,9730	0,9885	0,9529	0,9724	0,9684

- **LIAR-BR (pior grupo):** são alegações políticas curtas traduzidas do inglês. Decidir se são verdadeiras exige conhecimento de mundo que não está no texto, e o modelo fica perto do acaso.
- **Polígrafo:** o modelo acerta bem as verdadeiras, mas só metade das falsas. O dialeto pt-PT e o estilo de checagem portuguesa pesam (seção 4.6).
- **E-Farsas:** é o oposto; as poucas verdadeiras (62) são quase todas classificadas como fake (acerto de 40%).
- **Fake.br:** quase perfeito (0,97). São notícias completas, com sinais de estilo fortes, o que também sugere que o grupo é "fácil demais" para medir veracidade.

Grupos de rótulo constante (fora do pior grupo, mas informativos sobre o atalho): no subconjunto `true` (100% verdadeiro, estilo de alegação), o modelo acerta 60,6%. O encoder do pipeline antigo, sem DFR, acertava só 19%. Ou seja, o DFR reduziu, mas **não eliminou**, a tendência de chamar de fake tudo que tem cara de alegação de agência.

4.5 Recortes por canal e era

canal	n	% fake	acc	macro-F1
external_claim (LIAR/AVERITEC)	1.488	49,9	0,6499	0,6494
agency_claim (checadores)	8.477	91,5	0,8530	0,6951
whatsapp	957	37,3	0,7116	0,7096
covid	346	39,0	0,8121	0,8089
portal	1.081	56,5	0,9732	0,9726
press_true (manchetes, 100% true)	880	0,0	0,7420	—

Por era, o desempenho cai nos textos mais recentes: macro-F1 0,84 em ≤ 2017, 0,75 em 2018–2022 e 0,75 em ≥ 2023. Os textos novos vêm mais de checadores (mais fake, formato de alegação), que é o regime mais difícil.

4.6 Dialeto, casos limítrofes e grupos balanceados

recorte	n	resultado
pt-PT	2.461	macro-F1 **0,6893**
restante (pt-BR)	11.200	macro-F1 0,8057
casos limítrofes (enganoso, distorcido, fora de contexto)	529	acc 0,8166
falso puro	5.623	acc 0,8924
só grupos balanceados (51,9% fake)	5.536	acc 0,7314, macro-F1 0,7313, ECE 0,0215

- **O viés dialetal é grande:** 12 pontos de macro-F1 entre pt-PT e pt-BR. O BERTimbau foi pré-treinado em pt-BR.
- **Nos grupos balanceados**, que é onde a pergunta "o texto é verdadeiro?" está bem posta, o modelo fica em **0,73**. Esse é o número mais honesto sobre a capacidade real, sem a ajuda dos grupos fáceis.

4.7 Verificações de integridade

- O notebook de avaliação (`organizado/notebook/`) recalcula todas as métricas a partir das predições salvas, e **as 24 conferências bateram** com o `metrics.json` do treino.
- **Pesos:** reinferindo 300 textos do teste com os pesos salvos, a probabilidade calibrada mudou em média 0,0007, e a classe prevista foi a mesma em 100% dos casos.
- **Calibração:** aplicar `calibration.json` ao score salvo reproduz a probabilidade calibrada com erro de 1e-16.
- **Vazamento:** 0 `rid` em comum entre treino e teste, e 0 duplicatas normalizadas entre treino/validação e teste.

---

5. Principais desafios

#	desafio	como apareceu	como foi tratado
1	**Atalho de procedência**	43% do pool vem de grupos com rótulo constante; o prior da origem sozinho já dá acc 0,81	pesos DFR por célula, pior grupo como métrica-manchete, calibração nos grupos balanceados
2	**Desequilíbrio de classes** (2,75 fake : 1 true)	classe true sistematicamente pior	DFR (prior efetivo 0,60), macro-F1 como critério, métricas por classe no relatório
3	**Contaminação e vazamento**	30% do teste v6 estava no treino do v1; uma revisão encontrou **619 pares de quase-duplicatas (Jaccard > 0,8) entre treino e teste**	split por cluster MinHash, varredura exata pós-split (0 pares ≥ 0,8), comparação com o v1 também na parte limpa
4	**Duplicatas exatas**	473 cópias extras, 3 textos com rótulos conflitantes	primeiro agrupadas no mesmo lado do split; na versão final, removidas (uma linha por texto, 0 duplicatas restantes; conflitos resolvidos pela maioria ou, em empate, pelo menor `rid`)
5	**Rótulos fracos**	3.092 linhas `llm_local` (verificação por LLM local)	mantidos depois de medir: removê-los custou −0,027 de macro-F1 (T8)
6	**Calibração sob prior artificial**	calibrar na validação inteira (73% fake) desfazia o DFR; no pipeline antigo, o pior grupo caiu de 0,73 para 0,38	Platt ajustado só nos grupos balanceados
7	**Custo computacional**	o R0 em CPU levou cerca de 7 h (2 épocas); projeções para a T4 (Colab) eram hipóteses não medidas	treino local em GPU (RTX 3060, cerca de 11 min por run), batches por comprimento, bf16, congelamento de 6 camadas
8	**Ambiente Windows / Colab**	`--workers 2` quebrava (collator não serializável no `spawn`); a retomada pelo Drive não sobrevivia ao reinício da sessão	collators em nível de módulo; retomada procurando `last/` no Drive
9	**Experimentos enganosos**	na busca rápida, orçamento em passos e épocas diferentes inflaram um ganho de +0,17 que, controlado, era cerca de +0,04	revisão crítica adversarial, controles de 1 época, runs completos com 3 seeds
10	**Ruído entre seeds**	diferenças entre variantes da mesma ordem do ruído (0,004)	regra de decisão explícita: só vale um ganho acima do ruído e confirmado em 3 seeds

6. Aprendizados e lições

1. **Acurácia alta não prova que o modelo aprendeu a tarefa.** Um "modelo" que só olha a origem do texto chega a 0,81 de acurácia. Medir o **pior grupo** e as **métricas por classe** foi o que revelou o problema real.
2. **Os dados importam mais que os hiperparâmetros.** Dez variações de hiperparâmetro ficaram todas dentro do ruído. As maiores mudanças vieram de decisões sobre dados: remover rótulos fracos (−0,027), DFR, split por cluster, deduplicação.
3. **Vazamento é sutil.** Duplicatas exatas são fáceis de achar; quase-duplicatas (o mesmo boato com outra pontuação) não. Sem o split por cluster, parte do teste seria memória, e não generalização.
4. **Separar seleção de avaliação.** Escolher pela validação e abrir o teste só no fim evita "escolher o vencedor no teste". A validação foi dividida em duas (seleção e calibração) para que a calibração também não contamine a escolha.
5. **Calibração depende do prior.** Uma probabilidade só é útil se o prior em que ela foi calibrada for conhecido. Calibrar sob o prior artificial do corpus (73% fake) desfaz a correção de viés.
6. **Desconfiar dos próprios experimentos.** A revisão crítica da busca rápida derrubou conclusões que pareciam fortes. Unidades de orçamento (passos × tokens × épocas) precisam ser comparáveis.
7. **Ruído de seed é real.** Com amplitude de 0,004 entre seeds, qualquer ganho menor que isso é indistinguível de sorte. Repetir com 3 seeds custou tempo, mas evitou adotar "melhorias" falsas.
8. **Reprodutibilidade como requisito.** Seed fixa, cudnn determinístico e SHA-256 dos dados e do script permitiram refazer o run final e obter métricas idênticas.
9. **Empacotar para avaliação.** O notebook de avaliação roda em uma pasta autocontida (`organizado/notebook/`), falha explicitamente se faltar algum arquivo e confere as próprias métricas contra o treino.

---

7. Limitações e próximos passos

limitação	próximo passo
D1_dedup tem **só a seed 42**	rodar as seeds 43 e 44 e confirmar o empate (ou o ganho) sobre a BASE
Classe **true** fraca (F1 0,69; acerto de 40% no E-Farsas)	aumentar exemplos true verificados; testar perda focal ou reponderação por classe junto com o DFR
pt-PT 12 pontos abaixo	testar XLM-RoBERTa ou um modelo multilíngue; ponderar mais o Polígrafo
Generalização entre canais não medida neste pipeline	rodar o split `ood_wa` (treino sem WhatsApp, teste só WhatsApp)
LIAR-BR perto do acaso	as alegações exigem conhecimento externo; avaliar recuperação de checagens (ClaimReview) em vez de classificação pura
Quase-duplicatas entre 0,7 e 0,8 de Jaccard (219 pares treino×teste)	avaliar baixar o corte do cluster para 0,7
2 textos com rótulo conflitante empatado (1 fake × 1 true) foram decididos pelo menor `rid`, não por evidência	revisão manual desses 2 textos (`rid` 982 e 30447)
Licenças das fontes ainda em revisão	ver `docs/SOURCES_AND_LICENSES.md` antes de redistribuir dados ou modelo

---

8. Reprodução

Os resultados deste relatório são reproduzidos pela pasta `organizado/notebook/`. Ela é autocontida: tem o notebook de avaliação e todos os arquivos de que ele precisa, sem depender do restante do repositório. Pode ser copiada, zipada ou enviada ao Colab como está.

8.1 Conteúdo da pasta

```
notebook/
├── avaliacao_modelo_fakenewsbr_v6.ipynb notebook de avaliação
├── modelo/ model.safetensors (416 MB), config.json, tokenizer.json,
│    tokenizer_config.json, calibration.json (Platt a/b e limiares)
├── resultados/    predictions.csv (val_sel, val_calib e teste), metrics.json (do treino)
├── comparacao/    tfidf_predictions.csv, v1_predictions.csv,
│    v6_antes_dedup_predictions.csv (mesmo teste de 13.661 linhas)
└── dados/  v6_pool.parquet (pool deduplicado, 90.607 linhas)
```

Os 11 arquivos são **obrigatórios**. Se a pasta não for encontrada, ou se faltar qualquer arquivo, a primeira célula de código para com `FileNotFoundError` e lista o que está faltando.

8.2 Como executar

ambiente	passos
Local (VS Code / Jupyter)	abrir `avaliacao_modelo_fakenewsbr_v6.ipynb` de dentro de `notebook/` (ou de uma pasta que a contenha) e executar todas as células
Google Colab	zipar a pasta como `notebook.zip` (com a pasta `notebook/` dentro do zip), enviar para `/content` e executar todas as células; a primeira célula descompacta

Requisitos: Python 3 com `numpy`, `pandas`, `scikit-learn`, `matplotlib` e `pyarrow`. Para as seções de inferência (3 e 6), também `torch` e `transformers`. GPU é opcional: sem `torch`, essas duas seções são puladas com aviso, e as métricas (seções 4 e 5) continuam funcionando, porque são recalculadas a partir das predições salvas.

8.3 O que cada seção reproduz

seção do notebook	o que faz	relaciona-se com
0. Pasta	localiza `notebook/` e verifica os 11 arquivos	—
1. Métricas	funções copiadas do script de treino, incluindo as métricas de cada classe	seção 4.1
2. Calibração e modelo	lê `calibration.json` e carrega o modelo	seção 2.3
3. Inferência	classifica textos de exemplo com o Platt aplicado	—
4. Avaliação no teste	métricas globais e por classe nos limiares 0,5 e 0,46, conferência com o `metrics.json`, tabelas por grupo e por canal, diagrama de calibração	seções 4.2, 4.4 e 4.5
5. Comparação	maioria, TF-IDF, v1 antigo (inteiro e só a parte limpa), v6 antes do dedup e o modelo final	seção 4.3
6. Conferência dos pesos	reinfere 300 textos do teste com o modelo e compara com `predictions.csv`	seção 4.7

Resultado esperado: as 24 conferências com o `metrics.json` aparecem como `ok`, sem nenhuma `DIFERENTE`, e a seção 6 imprime `OK: pesos consistentes com predictions.csv` (classe prevista igual em 100% dos 300 textos).